



Algoritmos y Estructuras de Datos

1º Año

Ingeniería en Sistemas de Información

Funciones y Procedimientos Parámetros

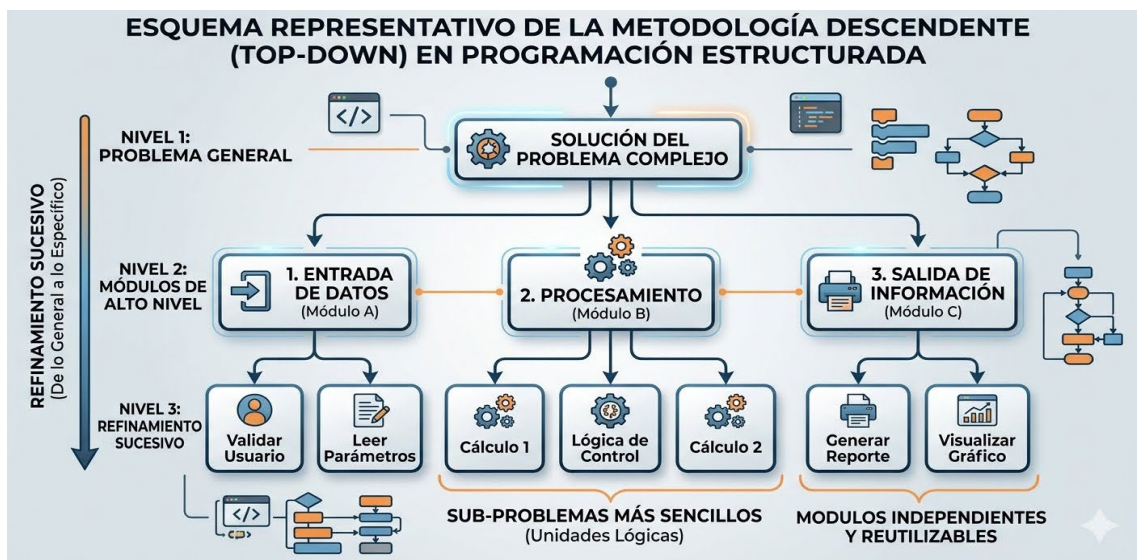
Cátedra Algoritmos y Estructuras de Datos UTN FRRO

Programación Descendente y Modularización

1. Diseño Descendente (Top-Down)

La especificación detallada de un problema establece su funcionalidad: **qué** debe hacer el programa, pero no **cómo** hacerlo.

Una vez establecidos los objetivos principales y las estructuras de datos necesarias, se inicia el diseño mediante el enfoque de **refinamiento sucesivo** o *Top-Down*. Este método consiste en dividir el problema central en varias tareas o niveles, generando una estructura de **módulos jerárquicos** donde cada módulo se comunica con su inmediato inferior mediante entradas y salidas de información.



A través de esta serie de descomposiciones sucesivas, cada etapa o módulo se subdivide en tareas más pequeñas. El proceso continúa hasta alcanzar los niveles inferiores, cuyas subtareas son lo suficientemente simples como para ser traducidas directamente a líneas de código (instrucciones simples).

La Prueba Ascendente (Bottom-Up)

En esta metodología es fundamental complementar el diseño con la **prueba ascendente**. Una vez finalizado el diseño conceptual, la verificación del software se realiza de abajo hacia arriba. Se prueban los módulos de los niveles inferiores utilizando datos simulados y se progresa hacia el programa conductor o principal para garantizar que el sistema integrado cumpla con el objetivo propuesto.

Objetivos Básicos de la Metodología Top-Down

- **Simplificación:** Descompone un problema complejo en niveles de refinamiento sucesivo más manejables.
- **Desarrollo independiente:** Permite que diferentes partes del problema sean programadas de forma paralela por distintos miembros de un equipo.
- **Reutilización de código:** Cada módulo puede ser invocado tantas veces como sea necesario, evitando la duplicación de código.
- **Flexibilidad:** El comportamiento de un módulo se adapta dinámicamente según los parámetros de entrada que recibe.
- **Mantenibilidad:** El programa final queda estructurado en bloques independientes, lo que facilita notablemente su lectura, depuración y mantenimiento.

2. Subprogramas y formas de Invocación

Los subprogramas son bloques de código aislados que resuelven una tarea específica dentro del sistema modular.

Características de la Invocación

- **Por Nombre:** Se ejecutan llamándolos explícitamente por el identificador asignado en su declaración.
- **Intercambio de Datos:** La comunicación entre el bloque invocante y el subprograma se realiza mediante **variables globales** y/o **parámetros**.
- **Diferenciación Teórica:** En la teoría de la programación, los subprogramas se clasifican en **Funciones** y **Procedimientos**. Su comportamiento y declaración difieren conceptualmente:

Característica	Funciones	Procedimientos
Retorno de datos	Devuelven un <i>único resultado</i> explícito mediante la sentencia return .	Realizan una acción. <i>Pueden devolver uno, varios o ningún resultado</i> a través de parámetros o variables globales .

Forma de Invocación	Se invocan dentro de expresiones (asignaciones, condicionales, salidas de datos o estructuras de decisión).	Se invocan como una sentencia propia e independiente utilizando su nombre.
Restricciones de llamada	No debe llamarse dentro de sí misma de manera descontrolada (evitar recursividad infinita).	Se ejecuta de forma lineal y devuelve el control al flujo principal al terminar.
Mecanismo de Parámetros	Teóricamente utiliza parámetros por valor .	Permiten el uso de parámetros por valor y por referencia .

Declaración de subprogramas en Diagrama de Chapin:

Posibles encabezados en cada diagrama de un subprograma:

Función suma (**p1**: tipodat, **p2**: tipodato, ..., **pn**: tipodato): **tipo_dato_función**

Procedimiento calculo (**nro**: tipodato, ..., **var porcentaje**: tipodato)

En la función siempre parámetros **por valor**.

var se antepone a los **parámetros por referencia o de entrada/salida, los que no se les antepone var, son por valor (ejemplo: nro)**

Los nombres de los subprogramas y parámetros, cumplen con las mismas reglas que los nombres de variables.

Nota sobre el Lenguaje Python:

En Python sólo existen las funciones, def ().

Por lo tanto, un procedimiento se implementa conceptualmente como una función que simplemente no retorna ningún o muchos valores (en forma de variables globales o a través de arreglos).

3. Tratamiento de Funciones en Python

Una función en Python es una estructura autónoma de programa. Se define mediante la palabra clave `def` y puede devolver valores al bloque invocante mediante la instrucción **`return`**.

```
# Python función
def nombre_funcion (parámetro_1, parámetro_2...parámetro_n):
    # Cuerpo de la función
    return resultado
```

Ejemplo de función:

```
# Python función
def suma(X): # declaración función
    s=0
    for i in range (10):
        s=s+X[i]
    return(s)

# prog_principal
A= [0.0]*10
# Carga un arreglo de 10 componentes de numeros reales

for i in range (10):
    A[i]=int (input ("ingrese valor de cada elemento "))
    Suma=suma(A) # invocación función
print ("la suma de los valores es ", Suma)
# Otra opción:
# print ("la suma de los valores es ", suma(A)) # invocación función
```

Particularidades en Python

- **Parámetros:** Se especifican entre paréntesis y separados por comas. No son obligatorios (existen funciones sin parámetros).
- **Ámbito y Anidamiento:** Las funciones pueden anidarse. Es importante gestionar correctamente el alcance de las variables locales y parámetros.
- **Versatilidad del return:**
 - Si se omite el `return`, la función actúa como un procedimiento.
 - Puede retornar un único objeto o múltiples valores simultáneamente pero dentro de una misma estructura de datos, por ejemplo, puede devolver un arreglo, lo cual una función, **DEVUELVE UN SOLO RESULTADO CON LA SENTENCIA RETURN.**
 - Pueden ubicarse múltiples sentencias `return` en puntos estratégicos del flujo

(por ejemplo, estructuras condicionales), evitando salidas abruptas en ciclos repetitivos.

Ejemplo 1: Función con múltiples puntos de retorno (Condicional)

```
# Python función
def es_par(numero):    # declaración
    if numero % 2 == 0:
        return True
    else:
        return False

# Programa Principal
nro=int(input('ingrese un número'))
# verificar si es par
respuesta=es_par(nro) # invocación función
if respuesta == True:
    print('El número es Par')
else:
    print('El número es Impar')
```

Ejemplo 2: Función que retorna múltiples valores.

El siguiente ejemplo dado que devuelve más de un valor en variables simples, es permitido en Python, pero en la cursada será tratado como Procedimiento devolviendo los resultados como variables globales o como parámetros por referencia.

```
# Python procedimiento

def cuadrado_y_cubo(numero):    # declaración procedimiento
    global cuadrado, cubo
    cuadrado = numero ** 2
    cubo = numero ** 3

# Prog_principal
nro=int(input('Ingrese un numero '))
# cálculo de cuadrado y cubo
cuadrado_y_cubo(nro) # invocación procedimiento
print('El cuadrado es ', cuadrado, ' y el cubo es ', cubo)
```

otra forma como Python función

```
def cuadrado_y_cubo(numero): # declaración procedimiento
    cuadrado = numero ** 2
    cubo = numero ** 3
    return cuadrado, cubo # R, retorna ambos valores"""

# Prog_principal
nro=int (input ('Ingrese un numero '))
# cálculo de cuadrado y cubo
cuadrado, cubo=cuadrado_y_cubo(nro) # invocación procedimiento
print ('El cuadrado es ', cuadrado, ' y el cubo es ', cubo)
```

4. Tratamiento de Procedimientos en Python

Un procedimiento se referencia escribiendo su nombre seguido de la lista opcional de parámetros. Al invocarse, el control del flujo se transfiere al inicio del procedimiento; una vez ejecutada la última instrucción, el control retorna automáticamente a la línea inmediata posterior a la que originó el llamado.

```
def procesar_datos (): # declaración procedimiento
    globales variables # Permite modificar una variable externa
    # Instrucciones del proceso
    print ("Proceso completado")
```

El pasaje de datos entre subprogramas se realiza a través del ámbito de Variables (Locales y Globales) o a través de Parámetros.

Ámbito de las Variables (Locales vs. Globales)

- **Variables Globales:** Son aquellas definidas en el módulo o programa principal. Son accesibles desde cualquier subprograma. Si un procedimiento necesita *modificar* su valor, debe declararse explícitamente dentro del módulo usando la palabra clave global.
- **Variables Locales:** Son aquellas declaradas e inicializadas dentro del subprograma. Su ciclo de vida está limitado a la ejecución del módulo; fuera de él no existen ni tienen valor.

El **ámbito de un identificador** (zona de vida de la variable) se limita estrictamente al bloque donde ha sido definido. Cuando un subprograma requiere procesar datos dinámicos sin depender de variables globales fijas, se recurre al uso de **parámetros**.

5. Gestión y Reglas de Parámetros

El pasaje de parámetros es el mecanismo formal para transferir información entre el bloque invocante y el subprograma. Los datos se vinculan entre un **parámetro actual o real** (el argumento enviado en la invocación) y un **parámetro formal o virtual** (la variable declarada en la cabecera del subprograma).

Reglas Fundamentales para la Transferencia de Parámetros

1. **Correspondencia en Cantidad:** El número de parámetros actuales en la invocación debe ser estrictamente igual al número de parámetros formales definidos en la cabecera del subprograma.
2. **Correspondencia de Tipo:** Cada parámetro actual debe ser del mismo tipo de dato (o compatible) que su parámetro formal correspondiente.
3. **Correspondencia de Orden:** La transferencia se realiza posicionalmente: el primer parámetro actual se asigna al primer parámetro formal, el segundo al segundo, y así sucesivamente.
4. **Declaración de Tipos:** En la cabecera del subprograma se define la estructura de los parámetros formales; no deben volver a declararse dentro del cuerpo del código como variables locales.

6. Clasificación de Parámetros según su Pasaje

A. Pasaje por Valor (Parámetros de Entrada)

Implica únicamente una transferencia de datos hacia el subprograma. El sistema realiza una copia del valor del parámetro actual en una dirección de memoria nueva asignada al parámetro formal.

Cualquier modificación que sufra el parámetro formal dentro del subprograma **no afecta** a la variable externa original en el programa principal.

En Python, las variables simples siempre son por valor (se devuelven por variables globales).

Las colecciones de datos como los arreglos, se comportan siempre por referencia. Toda modificación interna sobre sus elementos alterará la estructura original en el programa principal.

Ejemplo de Pasaje por Valor en Python

```
def cambio (x, y): # declaración procedimiento
    x = x + 25
    y = y - 3
    print ('Dentro del procedimiento mostrará: x:', x, ' y: ', y)

# Programa Principal
a = 5
b = 3
# Primer llamado
cambio (a, b) # Invocación, Muestra: Dentro de la función -> x: 30, y: 0
# Segundo llamado usando expresiones
cambio (a * 2, 7) # Invocación, Muestra: Dentro de la función -> x: 35, y: 4
# Verificación de las variables originales
print ('En el programa principal -> a: ', a, ' b: ', b) # Muestra: a: 5, b: 3
```

Las variables a y b mantienen sus valores originales (5 y 3) porque los tipos de datos simples (enteros) se pasan por valor de forma nativa en Python.

B. Pasaje por Referencia (Parámetros de Entrada/Salida)

Se utiliza para transferir información en ambas direcciones (ida y vuelta). En lugar de copiar el valor de la variable, el parámetro actual transfiere la **dirección real de memoria** donde se encuentra alojado el dato. El parámetro formal y el actual comparten exactamente el mismo espacio físico de memoria. Por este motivo, el parámetro actual **no puede ser una constante ni una expresión**; debe ser obligatoriamente una variable (un objeto mutable). Cualquier cambio realizado sobre el parámetro formal modificará de forma directa e inmediata a la variable externa del bloque invocante.

Ejemplo de Pasaje por Referencia en Python.

Comportamiento en Python con estructuras Arrays o Arreglos.

#Python

```
def cargo_arreglo(un_vector): # declaración Procedimiento
# devuelve el arreglo modificado con los valores ingresados
    for i in range (3):
        un_vector[i]=int (input ("ingrese cada elemento del arreglo: "))

def modificar_vector(un_vector): # declaración procedimiento
# Se modifica el primer elemento del arreglo recibido
    un_vector [0] = 99
```

```
def muestro_arreglo(un_vector): # declaración procedimiento
    for i in range (3):
        print(mi_arreglo[i])

# Programa Principal
mi_arreglo= [0]*3
cargo_arreglo(mi_arreglo) # invocación procedimiento
print ("Arreglo original ")
muestro_arreglo (mi_arreglo) # muestra arreglo original ejemplo mi_arreglo = [10, 20, 30]
modificar_vector(mi_arreglo) # invocación procedimiento
print ("Arreglo Modificado")
# Muestra elemento por elemento, podemos darle formato
muestro_arreglo(mi_arreglo) # Muestra: [99, 20, 30]
# muestra elemento por elemento en forma rápida
print (mi_arreglo [:])
```

Las modificaciones realizadas dentro de la función quedaron reflejadas de manera permanente en la variable de origen mi_arreglo.